Jun 14, 2025 at 10:29 PM

You said:

middleware and actionfilter in .net write a blog for interview preparation

ChatGPT said:

Here's a blog-style explanation on **Middleware vs Action Filters in .NET**, tailored for **interview preparation**, with diagrams, real-world examples, and code samples.

🔍 Middleware vs Action Filter in .NET – Explained for Interviews

When preparing for a .NET backend interview, understanding **Middleware** and **Action Filters** is essential, especially for roles involving API development, cross-cutting concerns, or ASP.NET Core pipelines. Though they sound similar, they serve different purposes and operate at different layers of the request pipeline.

🧱 What is Middleware?

👉 Definition:

Middleware is a **component in the ASP.NET Core pipeline** that processes HTTP requests and responses globally. Every request passes through a series of middleware components before reaching the controller.

📍 Use Cases:

- Logging
- Authentication
- CORS
- Exception Handling
- Request/Response modification

⚙️ How Middleware Works

```
Request --> Middleware1 --> Middleware2 --> ... --> Endpoint (Controller)
                                    ↑
                        Response passes back through
```

✅ Middleware Example

C#

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;

    public LoggingMiddleware(RequestDelegate next) => _next = next;

    public async Task Invoke(HttpContext context)
    {
        Console.WriteLine($"Request Path: {context.Request.Path}");
        await _next(context); // Call the next middleware
        Console.WriteLine($"Response Status: {context.Response.StatusCode}");
    }
}

// Register in Program.cs or Startup.cs
app.UseMiddleware<LoggingMiddleware>();
```

🎯 What is an Action Filter?

👉 Definition:

Action Filters are attributes that allow you to **execute logic before and after controller actions** run. They operate **only at the MVC layer** (Controllers & Razor Pages).

📍 Use Cases:

- Model validation
- Logging
- Authorization (fine-grained)
- Caching
- Modifying Action Results

⚙️ Filter Pipeline

```
Request --> Middleware --> Routing --> Action Filter --> Action Method --> Result Filter
```

✅ Action Filter Example

C#

```
public class LogActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine("Before action executes");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("After action executed");
    }
}

// Apply globally
services.AddControllers(config =>
{
    config.Filters.Add<LogActionFilter>();
});
```

Or apply to a specific controller/action:

C#

```
[ServiceFilter(typeof(LogActionFilter))]
public class HomeController : Controller
{
    public IActionResult Index() => View();
}
```

🔍 Middleware vs Action Filter – Side-by-Side Comparison

Feature	Middleware	Action Filter
Scope	Global	Controller/Action only
Layer	HTTP pipeline	MVC pipeline
Use for	Logging, Auth, CORS, Error Handling	Validation, Result Wrapping

Feature	Middleware	Action Filter
Order/Sequence	Registered order in `Program.cs`	Based on filter pipeline (global > controller > action)
Access to HttpContext?	✅ Yes	✅ Yes (via `context.HttpContext`)
Can short-circuit?	✅ Yes (`return`)	✅ Yes (set `context.Result`)
Performance	Runs on all requests	Runs on MVC requests only

🧠 Interview Tips

✅ When to use Middleware?

- If you want to apply logic **globally** for **all requests**, including non-MVC ones (e.g., static files).
- Good for low-level concerns like setting headers, exception logging, etc.

✅ When to use Action Filter?

- When you want to **decorate or intercept controller logic**.
- Best for handling things **close to the controller**, like input validation or result formatting.

🔄 Can They Be Used Together?

Yes! Middleware and Action Filters **complement each other**.

Example scenario:

- Use middleware for logging and exception handling
- Use an action filter to validate model state and log method execution time

---

🧪 Bonus: Custom Action Filter for Model Validation

C#

```
public class ValidateModelAttribute : ActionFilterAttribute
{
```

```
    public override void OnActionExecuting(ActionExecutingContext context)
    {
        if (!context.ModelState.IsValid)
        {
            context.Result = new BadRequestObjectResult(context.ModelState);
        }
    }
}
```

Apply like this:

```
C#

[ValidateModel]
public IActionResult Create([FromBody] ProductDto dto)
{
    // Safe to assume ModelState is valid
}
```

📌 Conclusion

Middleware	Action Filter
Used for **global, cross-cutting** concerns	Used for **controller-specific** logic
Operates at **HTTP pipeline level**	Operates at **MVC level**
Registered in `Program.cs` via `Use...`	Added via attributes or global filter registration

---

🧑‍💻 Final Thought

In interviews, be ready to:

- Explain the request pipeline
- Differentiate between middleware and filters
- Give real use cases where you used both

> 🔁 Always remember: Middleware is global, Filters are MVC-specific.

If you'd like, I can provide a **diagram** of the ASP.NET Core pipeline showing middleware and filters together. Just ask!

Today 1:48 PM

You said:

make mark down, avoid duplicate

ChatGPT said:

This response couldn’t load

You said:

make mark down, avoid duplicate

ChatGPT said:

This response couldn’t load